POLCOVID · CHEST X-RAY CLASSIFICATION

Tests, results, and graphs

A user guide to running the software tests, executing the experiments, and turning prediction files into summary tables and figures.

Repository path: the project root (the directory that contains `src/`, `scripts/`, `configs/`, and `tests/`).
Companion documents: `README.md`, `docs/analysis_plan.md`.
October 2026

Research code for a controlled comparison of vision-language models and supervised CNNs. Not a medical device.

Contents

1 Three ways to get results

2 What the experiments measure

3 Install the environment

4 Unit tests and lint

5 Pipeline test on synthetic predictions

6 Audit, power analysis, and legacy numbers

7 Run an experiment

8 Where the result data is written

9 Build the summary tables

10 Build the graphs

11 Build the manuscript tables

12 Read a result by hand

13 Run the GPU suite in Colab

14 Keep the test set honest

15 Troubleshooting

Commands in this guide are run from the repository root, with the virtual environment activated. A leading `python` means the interpreter in that environment.

1. Three ways to get results

There are three separate jobs in this repository. They share code, and they write different files.

A **Check that the code works.** `pytest` runs the CPU unit tests. A second, optional step writes fake predictions and drives the analysis, table, and figure scripts, which is what continuous integration does. Neither step is a scientific result.

B **Turn data that is already in the repository into tables and graphs.** The audit, the power analysis, the legacy notebook numbers, and the linear-probe predictions are committed under `results/`. `scripts/analyze.py` and `scripts/make_figures.py` read those files. No GPU is required, and the image datasets are not downloaded.

C **Run a model, then make graphs from its predictions.** `scripts/run_experiment.py` writes one directory per run. After a `--final` run, the same analysis and figure scripts add that run to the summaries and graphs.

Track A — software tests

```
python -m venv .venv && . .venv/bin/activate
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
pip install -r requirements-dev.txt
ruff check src scripts tests
pytest -q
```

Track B — graphs from committed results

```
python scripts/analyze.py
python scripts/make_figures.py
python scripts/make_tables.py
```

Outputs land in `results/summary/`, `paper/figures/`, and `paper/generated/`. Figures whose experiments have not been run are skipped. Tables for those experiments say “pending”. With only the committed linear probes, you should get a data-efficiency figure and a ROC panel for the supervised family, plus the audit contact sheets.

Track C — one real experiment, then its graphs

```
# Validation only. Does not touch the test split.
python scripts/run_experiment.py --config configs/cnn_resnet50.yaml

# Score the test split as well, and append to the test-access log.
python scripts/run_experiment.py --config configs/cnn_resnet50.yaml --final

python scripts/analyze.py
python scripts/make_figures.py
python scripts/make_tables.py
```

> **Start here if you only want the pictures.** Track B is enough to see the figure pipeline working. Track C is what fills in the missing panels. Section 10 lists every graph and the file it comes from.

2. What the experiments measure

Each experiment is a binary classification of chest radiographs from the published POLCOVID splits. Both tasks are exactly balanced: 1,596 training images, 228 validation images, and 456 test images.

Task	Negative class (label 0)	Positive class (label 1)	Hugging Face dataset
`task1`	Normal	Pneumonia	`Henryayay/POLCOVID_task1_70_10_20`
`task2`	Non-COVID Pneumonia	COVID-19	`Henryayay/POLCOVID_task2_70_10_20`

The Hub revisions are pinned in `src/cxr/tasks.py`. Every run records the revision it actually loaded in `meta.json`.

The primary endpoint is the area under the ROC curve (AUROC) of the positive class on the test set. Secondary endpoints are balanced accuracy, sensitivity, specificity, F1, Matthews correlation, AUPRC, Brier score, expected calibration error, and the positive-prediction rate. The decision threshold for the main tables is the one that maximises Youden's J on the validation split. Calibration comparisons use the argmax rule, which is `logit ⩾ 0`.

Five families of model are configured under `configs/`:

- **CNNs** (`kind: cnn`): ResNet-50, DenseNet-121, and a torchxrayvision DenseNet-121, trained from the YAML recipe.
- **Linear probes** (`kind: linear_probe`): a logistic regression on frozen RAD-DINO or ImageNet ResNet-50 embeddings.
- **Zero-shot VLMs** (`kind: vlm_zeroshot`): Gemma-3-4B-it and MedGemma-4B-it, scored by class-name likelihood.
- **In-context learning** (`kind: vlm_icl`): the same models with *k* exemplars, either random or RAD-DINO nearest neighbours.
- **LoRA** (`kind: vlm_lora`): the same models fine-tuned with LoRA, including a Gemma-3 data-efficiency curve.

The score stored for every image is a **logit for the positive class**. For a vision-language model that logit is the log-likelihood of the positive class name minus the log-likelihood of the negative class name. Probability is the sigmoid of the logit. Hard labels for the argmax rule are `logit ⩾ 0`.

3. Install the environment

Python 3.10 or newer is required. The continuous-integration job uses 3.12.

CPU machine

Use this for unit tests, the audit, the power analysis, analysis, tables, and figures. Installing PyTorch from the CPU index first stops `pip` from pulling a CUDA wheel.

```
python -m venv .venv
. .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
pip install -r requirements-dev.txt
```

`requirements-dev.txt` includes `requirements.txt` plus `pytest` and `ruff`. Core packages are NumPy, pandas, PyArrow, SciPy, scikit-learn, Hugging Face `datasets` and `transformers`, Pillow, PyYAML, and Matplotlib.

GPU machine

An A100 is the machine the vision-language configs were written for. CNN and linear-probe runs also use CUDA when it is present.

```
pip install -r requirements.txt -r requirements-gpu.txt
```

`requirements-gpu.txt` adds Unsloth, TRL, PEFT, Accelerate, torchxrayvision, and OpenCLIP. Unsloth pulls a compatible bitsandbytes build for 4-bit loading where a config asks for it. The committed VLM configs set `load_in_4bit: false` on the LoRA runs.

Hugging Face access

- The POLCOVID datasets and Gemma-3 weights are loaded from the Hub on first use. They are cached by the `datasets` and `transformers` libraries (usually under `~/.cache/huggingface`).
- MedGemma is gated. Accept the Health AI Developer Foundations terms on the model page, then export a token: `export HF_TOKEN=...`. In Colab, store that token as a secret named `HF_TOKEN`.
- Nothing in this repository pushes to the Hub.

> **Disk.** Checkpoints (`model.pt`, LoRA `adapter/` directories) and the embedding cache (`cache/*.npy`) are git-ignored. They can be large. Prediction parquet files and `meta.json` are the files you keep and, when a run is final, commit.

4. Unit tests and lint

The unit tests live in `tests/` and do not need a GPU, a download of the POLCOVID images, or a finished experiment. Pytest is configured in `pyproject.toml`: `src/` and `tests/` are on the Python path, and the test path is `tests`.

```
pytest -q
pytest -q --tb=short                # full tracebacks for failures
pytest tests/test_metrics_stats.py  # one module
pytest -q -k "delong or holm"       # tests whose names match
ruff check src scripts tests
```

A quiet run prints a dot per test and a summary line. Exit code 0 means every test passed. Ruff checks formatting and import rules in `src`, `scripts`, and `tests`. It does not look at `legacy/` or `notebooks/`.

What each test module covers

File	What a failure there means
`tests/test_components.py`	Prompts, label parsing, class-stratified subsets, image padding and hashing, the six robustness perturbations, random and nearest-neighbour exemplar selection, and the duplicate detector used by the audit.
`tests/test_metrics_stats.py`	Threshold metrics against scikit-learn, calibration (ECE, temperature scaling, Youden threshold), Wilson intervals, DeLong AUROC tests, exact McNemar, the paired bootstrap, and Holm correction.
`tests/test_vlm_scoring.py`	Likelihood scoring on a tiny random Gemma-3 that uses the real processor: the prefix cache matches a full forward pass, and the chat template contains a single beginning-of-sequence token. The processor config is downloaded from `unsloth/gemma-3-4b-it` at the pinned revision. If that download fails, the module skips itself rather than failing.

To run the suite without any Hub access:

```
pytest -q --ignore=tests/test_vlm_scoring.py
```

`tests/tiny_gemma3.py` is a helper, not a test. It builds a randomly initialised Gemma-3 with a handful of layers so the scoring tests stay on CPU.

What continuous integration runs

`.github/workflows/ci.yml` runs on every push and pull request, on Ubuntu with Python 3.12:

1. Install CPU PyTorch, then `requirements-dev.txt`.
2. `ruff check src scripts tests`.
3. `pytest -q`.
4. The synthetic pipeline test in the next section, with 200 bootstrap replicates.
5. `python scripts/make_tables.py`, then `git diff --exit-code paper/generated`. A pull request fails if the committed LaTeX tables are out of date relative to `results/`.

CI does not train a model and does not call `--final` on real data.

5. Pipeline test on synthetic predictions

This is the way to prove that analysis, tables, and figures run, without waiting for a model and without writing anything into the paper. `scripts/make_synthetic_results.py` builds a results tree with the same layout as a real run and plants a file named `SYNTHETIC` at its root. The numbers are random logits with a planted class separation. They are not experimental results.

```
python scripts/make_synthetic_results.py --out /tmp/synth
cp -r results/audit /tmp/synth/audit
python scripts/analyze.py --results /tmp/synth --n-boot 200
python scripts/make_tables.py --results /tmp/synth --out /tmp/synth_tables
python scripts/make_figures.py --results /tmp/synth --out /tmp/synth_figs
ls /tmp/synth_figs
```

Defaults are 3 seeds, 60 validation images, and 120 test images per task (`--seeds` , `--n-val` , `--n-test`). The real plan uses 10,000 bootstrap replicates; `--n-boot 200` is only for this smoke test. Use the full default when you analyse real predictions.

> **The paper directory is protected.** If the results root contains a file named `SYNTHETIC` , `make_tables.py` and `make_figures.py` refuse to write into `paper/` . Point `--out` somewhere else, as the commands above do. Do not copy a `SYNTHETIC` tree over `results/` .

After a successful synthetic run you should see, under `/tmp/synth_figs` , some or all of `fig_icl_shots.pdf` , `fig_roc_reliability.pdf` , `fig_efficiency.pdf` , and `fig_robustness.pdf` , each also saved as a 200 dpi PNG, plus copies of the audit contact sheets. Under `/tmp/synth/summary` you should see the CSVs described in section 9. Open `comparisons.csv` and confirm the `tier` column contains `primary` and `secondary` .

Optional CNN smoke run

`configs/smoke_cnn.yaml` is a two-epoch ResNet-18 on 64×64 images, task 1 only, seed 0. It is a wiring check, not a result.

```
python scripts/run_experiment.py --config configs/smoke_cnn.yaml --limit 48 --final
```

`--limit 48` keeps the first 48 images of each split and redirects output to `results/_smoke/` , which is git-ignored. `analyze.py` ignores any run whose `meta.json` has a non-zero `limit` , so these predictions cannot enter the test-set tables even if you point the script at them. The first run downloads the task 1 dataset.

6. Audit, power analysis, and legacy numbers

These three scripts do not train a model. Their outputs are already committed. Re-run them when you want to regenerate the files from scratch.

Dataset audit

```
python scripts/audit_dataset.py
# or: python scripts/audit_dataset.py --out results/audit
```

This downloads both datasets and writes `results/audit/`. It is the leakage and shortcut audit: perceptual hashes, near-duplicate pairs, cross-task copies, and simple image statistics that can act as shortcuts.

File	Contents
`records_task1.parquet`, `records_task2.parquet`	One row per image: split, index, label, SHA-256, perceptual hash, geometry, intensity.
`duplicates_task1.csv`, `duplicates_task2.csv`	Graded pairs inside a task (exact, near-identical, highly similar).
`cross_task_duplicates.csv`	Graded pairs between task 1 and task 2.
`class_summary_task1.csv`, `class_summary_task2.csv`	Per split and class, medians of geometry and intensity.
`top_pairs_task1.png`, `top_pairs_task2.png`	Contact sheet of the most similar cross-split pairs. These are the audit graphs.
`audit_summary.json`	Headline counts used by the paper tables.

The leakage sensitivity in `analyze.py` drops test images whose correlation with a training image is at least 0.95, using these files. If `results/audit/` is missing, that CSV is empty and the leakage table stays pending.

Power analysis

```
python scripts/power.py
```

The test set size is fixed (456 images, 228 per class), so this script reports what that sample can resolve rather than choosing a sample size. It reads the primary comparisons from `configs/analysis_plan.yaml` and writes `results/power/`:

- `accuracy_precision.csv` — Wilson 95% CI half-width of a proportion, at several true rates.
- `auroc_precision_mde.csv` — Hanley–McNeil standard error, CI half-width, and the AUROC difference detectable at 80% power, for correlations 0.3, 0.5, and 0.7, at $\alpha = 0.05$ and at the Holm worst-case α.
- `mcnemar_mde.csv` — smallest paired accuracy difference detectable by McNemar's test, by discordant-pair rate.
- `power.json` — the same three tables in one file.

The script prints the rounded tables to the terminal. There is no graph; `make_tables.py` turns `power.json` into `paper/generated/tab_power.tex`.

Legacy notebook numbers

```
python scripts/extract_legacy_results.py
```

Parses stored cell outputs under `legacy/` (it does not re-execute the notebooks) and writes `results/legacy/legacy_results.csv`. Columns include task, family (`cnn`, `finetune`, `icl_base`, `icl_nshot`), notebook path, shot count, accuracy percent, per-class recall, and a text field of known protocol issues. Treat these rows as preliminary. They are not used as inputs to the pre-registered comparisons.

7. Run an experiment

Every scientific run is one YAML file in `configs/` passed to `scripts/run_experiment.py`.

```
python scripts/run_experiment.py --config configs/probe_imagenet_resnet50.yaml
python scripts/run_experiment.py --config configs/probe_imagenet_resnet50.yaml --final
```

Without `--final`, only the validation split is scored. Model selection and learning-rate sweeps use validation loss or validation negative log-likelihood. They never see the test split. With `--final`, the test split is scored as well, configured perturbations are applied to the test images, and one JSON line is appended to `results/test_access_log.jsonl`.

Command-line flags

Flag	Default	Effect
`--config`	required	Path to the experiment YAML.
`--final`	off	Also score the test split and log test access.
`--tasks`	the YAML list	Restrict to `task1`, `task2`, or both.
`--results`	`results`	Root directory for prediction files.
`--cache`	`cache`	Where frozen-encoder embeddings are stored as `.npy`.
`--limit N`	0 (all images)	Smoke test: first N images per split. Output goes to `results/_smoke/` and is excluded from test-set analysis.
`--only-runs`	all runs	Only these run directory names, for example `seed0` `seed1` or `seed0_shots4`.
`--overwrite`	off	Recompute a run even when `meta.json` already exists.
`--device`	`cuda` if available, else `cpu`	For example `--device cpu`.

How a run is named

The directory name is built from the seed, the training fraction, and the shot count:

```
seed{seed}
seed{seed}_frac{fraction}     # only when the fraction is not 1
seed{seed}_shots{k}           # only for in-context learning
seed{seed}_frac{fraction}_shots{k}
```

Examples: `results/task1/cnn_resnet50/seed3/`, `results/task2/probe_rad_dino/seed0_frac0.25/`, `results/task1/vlm_gemma3_icl_knn/seed2_shots8/`. A finished run contains `predictions.parquet` and `meta.json`. Training runs also keep `train_log.json` and a checkpoint (`model.pt` or `adapter/`). CNN experiments that sweep the learning rate write `hparam_selection.json` once, next to the run directories.

Runs are resumable

If `meta.json` already exists, the run is skipped, with two exceptions. Passing `--overwrite` always recomputes it. Passing `--final` recomputes a run whose existing metadata says `"final": false`, so you can train on validation and later score the test set. Checkpoints are reused: a CNN loads `model.pt` if it is there, and LoRA skips training when `adapter/` is there. You do not retrain just to add the test split.

The log line `skip task1/cnn_resnet50/seed0 (exists)` means that run was left untouched. `done seed0 in 1234s` means it finished and wrote files.

Recommended order

Later configs reuse earlier choices and caches. Run them in this order, first without `--final` while you are developing, then again with `--final` once the analysis plan is frozen:

1. `cnn_resnet50` — selects the learning rate on a sweep seed and writes `hparam_selection.json`.
2. `cnn_resnet50_data_efficiency` — reads that learning rate (`lr_from: cnn_resnet50`). It fails with `FileNotFoundError` if the ResNet-50 run has not been done.
3. `cnn_densenet121`, then `cnn_xrv_densenet121`.
4. `probe_rad_dino` — also fills `cache/task*_rad_dino_*.npy`, which retrieval ICL reuses.
5. `probe_imagenet_resnet50` — CPU-capable; predictions for seed 0 are already in the repository.
6. `vlm_gemma3_zeroshot` and `vlm_medgemma_zeroshot`.
7. `vlm_*_icl_random`, then `vlm_*_icl_knn`.
8. `vlm_*_lora`, then `vlm_gemma3_lora_data_efficiency`.

Retrieval ICL will compute RAD-DINO embeddings itself if the cache is empty, so the probe does not have to come first, but sharing `--cache` avoids doing that work twice. Colab points both at the same Drive folder; locally the default `cache/` directory is shared as long as you do not override it.

Config catalogue

Config	Kind	What varies
`cnn_resnet50.yaml`	CNN, ResNet-50	Seeds 0–4. Learning rate chosen from 1e-5, 1e-4, 3e-4. All six test perturbations.
`cnn_resnet50_data_efficiency.yaml`	CNN, ResNet-50	Seeds 0–4 at 10%, 25%, 50% of the training set. Learning rate inherited.
`cnn_densenet121.yaml`	CNN, DenseNet-121	Seeds 0–4. Same perturbation list.
`cnn_xrv_densenet121.yaml`	CNN, CXR-pretrained DenseNet-121	Seeds 0–4. Needs torchxrayvision.
`probe_rad_dino.yaml`	Linear probe	Seed 0 at 10%, 25%, 50%, 100% of training data. Encoder `microsoft/rad-dino`.
`probe_imagenet_resnet50.yaml`	Linear probe	Same fractions, ImageNet ResNet-50 features. Seed 0 is already committed for both tasks.
`vlm_gemma3_zeroshot.yaml`	Zero-shot VLM	One run, seed 0. Also generates free text for the parsing comparison.
`vlm_medgemma_zeroshot.yaml`	Zero-shot VLM	Same, MedGemma. Requires the Hub terms and `HF_TOKEN`.
`vlm_gemma3_icl_random.yaml`	Random ICL	Seeds 0–4 × k in {2, 4, 6, 8, 10}. Each seed is an independent exemplar draw.
`vlm_gemma3_icl_knn.yaml`	Retrieval ICL	Same grid. Exemplars are class-balanced RAD-DINO neighbours; seeds change order.
`vlm_medgemma_icl_random.yaml`, `vlm_medgemma_icl_knn.yaml`	ICL	Same grids on MedGemma.
`vlm_gemma3_lora.yaml`, `vlm_medgemma_lora.yaml`	LoRA	One run, seed 0, on the full training set. The seed fixes initialisation, dropout and shuffle. Six test perturbations, free-text generation on the clean images.
`vlm_gemma3_lora_data_efficiency.yaml`	LoRA	Seed 0 at 10%, 25%, 50% of training data. The seed also fixes which class-stratified subset is used.
`smoke_cnn.yaml`	CNN smoke test	Not a result. See section 5.

`configs/analysis_plan.yaml` is not an experiment. It is the machine-readable analysis plan that `analyze.py` executes. The prose version is `docs/analysis_plan.md`. Where they differ, the YAML wins.

Narrow a long run

```
# One task, two seeds, validation only.
python scripts/run_experiment.py \
  --config configs/cnn_resnet50.yaml \
  --tasks task1 \
  --only-runs seed0 seed1

# One ICL cell.
python scripts/run_experiment.py \
  --config configs/vlm_gemma3_icl_random.yaml \
  --tasks task2 \
  --only-runs seed0_shots4 \
  --final
```

8. Where the result data is written

A finished tree looks like this. Names in brackets appear only for some experiment kinds.

```
results/
  test_access_log.jsonl          # one line per --final run
  audit/                         # section 6
  power/                         # section 6
  legacy/legacy_results.csv
  summary/                       # section 9, created by analyze.py
  gradcam/                       # section 10, created by gradcam.py
  task1/
    cnn_resnet50/
      hparam_selection.json
      seed0/
        predictions.parquet
        meta.json
        train_log.json
        model.pt                 # git-ignored
    probe_rad_dino/seed0_frac0.1/
    vlm_gemma3_icl_knn/seed0_shots4/
    vlm_gemma3_lora/seed0/adapter/   # git-ignored
  task2/...
```

predictions.parquet

One row per image per perturbation. Required columns, present for every model:

Column	Meaning
`split`	`validation` or `test`. Training images are not scored into this file.
`index`	Row index inside that split of the published dataset.
`y_true`	0 or 1. 1 is the positive class of the task.
`logit`	Score for the positive class. Higher means more positive. This is the column metrics use.
`perturbation`	`none` on clean images. On the test split, also `border_mask`, `centre_mask`, `gamma_low`, `gamma_high`, `noise`, and `jpeg` when the config lists them.

Vision-language rows add:

Column	Meaning
`logp_neg`, `logp_pos`	Log-likelihood of each class name. `logit` is `logp_pos - logp_neg`.
`logit_cf`	Content-free logit (the same prompt on a blank image), used for contextual calibration. Missing when the config sets `content_free: none`.
`generated`	Free-text answer, only when `also_generate: true` and the perturbation is `none`.
`parsed_label`	0 or 1 if the text matched a class name, −1 if it did not, −100 if generation was not run.

meta.json

Provenance for that directory. Fields you will use most often:

- `task`, `experiment`, `run`, `kind`, `seed`, `train_fraction`, `shots`.
- `final` — true only when the command included `--final`. Test-set metrics require this.
- `limit` — non-zero for smoke runs. Those runs are dropped from test-set analysis.
- `config` and `config_hash` — the YAML and a 12-character hash of it.
- `git_commit` — `HEAD` at runtime, with a `-dirty` suffix if the worktree had uncommitted changes.
- `dataset_id`, `dataset_revision`, `environment` (Python, package versions, GPU name), `wall_seconds`.

- Model-specific fields: chosen learning rate and epoch for CNNs, inverse-regularisation `C` for probes, exemplar indices for random ICL.

test_access_log.jsonl

Append-only. Each line is one `--final` evaluation: UTC time, task, experiment, run name, config hash, git commit, hostname, and user. Compare its timestamps with `git log docs/analysis_plan.md` to see that the analysis plan was committed before the test split was scored. Do not rewrite earlier lines.

Embedding cache

Files are `cache/{task}_{encoder}_{split}.npy`, and `cache/{task}_{encoder}_test_{perturbation}.npy` for perturbed test embeddings. Encoders in use are `rad_dino` and `imagenet_resnet50`. Deleting a `.npy` file just causes it to be recomputed.

9. Build the summary tables

```
python scripts/analyze.py
python scripts/analyze.py --results results --plan configs/analysis_plan.yaml
python scripts/analyze.py --n-boot 10000          # the plan default
python scripts/analyze.py --out results/summary   # default is <results>/summary
```

The script loads every `predictions.parquet` three directory levels under the results root (`task/experiment/run`). It ignores runs that are not `final` or that used `--limit` when it computes test-set metrics. Validation metrics are still computed for those runs. It prints how many runs it loaded, then writes `results/summary/`.

File	One row is	Use it for
`run_metrics.csv`	One run, one score variant, one decision rule, one split	Per-seed numbers, confidence intervals, the exact threshold that was used.
`condition_summary.csv`	Mean, standard deviation, min, and max across repeated runs. A single run has no standard deviation.	The curves and the "mean ± SD" cells in the paper. LoRA, probes and zero-shot are one number.
`selections.json`	The validation-best member of each family, per task	Which condition the primary comparisons actually used. Ties break alphabetically, then toward smaller *k*.
`comparisons.csv`	One pre-registered comparison on one task	Bootstrap difference and CI (`boot_diff`, `boot_ci_low`, `boot_ci_high`), `boot_p`, Holm-adjusted `p_holm` on the primary rows, plus `delong_seedmean_p` and `mcnemar_run0_p`.
`robustness_runs.csv`, `robustness.csv`	Per run, then mean across runs	Test AUROC under each perturbation and the drop from the clean image.
`leakage_sensitivity.csv`	One condition	Test AUROC with and without test images that have a highly similar training image.
`generation_vs_likelihood.csv`	One VLM run and split	Parse-failure rate, accuracy of the parsed string, accuracy of the likelihood score, and agreement.
`summary.json`	The whole analysis	Selections, ensemble weights, and comparisons in one file.

Score variants and decision rules

Filter on these columns before you plot or quote a number. Mixing them double-counts a run.

Column	Values	When it applies
`variant`	`raw`	Every model. This is the primary AUROC.
`variant`	`val_centered`	VLMs. Subtracts the validation-set mean logit, which removes a constant preference for one label.
`variant`	`contextual`	VLMs that stored `logit_cf`. Subtracts the content-free logit.
`rule`	`default`	Threshold 0 (argmax). Used for calibration comparisons.
`rule`	`youden_val`	Threshold maximising Youden's J on validation. Used for the main threshold metrics.
`split`	`validation` or `test`	Quote `test` for the paper. Use `validation` only to see what selection saw.

Metric columns on `run_metrics.csv` include `auroc`, `auroc_ci_low`, `auroc_ci_high` (DeLong), `auprc`, `accuracy`, `balanced_accuracy`, `sensitivity`, `specificity`, `f1`, `mcc`, `brier`, `ece`, `ece_temp_scaled`, `positive_rate`, `threshold`, and `n`. On `condition_summary.csv` each metric grows suffixes `_mean`, `_std`, `_min`, and `_max`, plus `n_runs`.

Shots are stored as the integer *k*, or as −1 when the experiment has no exemplars (CNNs, probes, zero-shot, LoRA). Training fraction is 1.0 for a full-data run.

What the comparisons file contains

Primary comparisons, each run on both tasks and then Holm-adjusted across all ten tests, are declared in `configs/analysis_plan.yaml`:

- `best_vlm_vs_best_supervised` — AUROC.
- `medgemma_vs_gemma3_lora` and `medgemma_vs_gemma3_zeroshot` — AUROC.
- `icl_knn_vs_random_gemma3` — AUROC at the validation-best *k* of each strategy.
- `calibration_valcentered_vs_raw_gemma3_icl_random` — balanced accuracy at argmax.

Secondary rows are exploratory and are not Holm-adjusted: the same ICL contrast for MedGemma, contextual calibration, val-centring of Gemma-3 zero-shot, Gemma-3 LoRA against the best supervised model, and a validation-weighted ensemble of the best VLM and the best supervised model against each of its parts. The ensemble is built inside `analyze.py`. It is not a directory written by `run_experiment.py`. Its weight is in `summary.json` under `ensembles`. If one component has a single run and the other has several, that single run is paired with every run of the other component. Equal counts are paired in sorted run-name order.

A comparison row with `status` other than `ok` means a condition has not been run with `--final` yet. That is expected until Track C is complete. The bootstrap uses 10,000 paired, class-stratified resamples of the test images and seed 20261011. On the full suite this step is the slow part of `analyze.py`; it does not use the GPU.

10. Build the graphs

```
python scripts/make_figures.py
python scripts/make_figures.py --results results --out paper/figures
ls paper/figures
```

The script reads `results/summary/`, so run `analyze.py` first. Each plot is written twice: a vector `.pdf` for the manuscript and a 200 dpi `.png` for viewing. A figure is skipped, rather than drawn empty, when its inputs are missing. The manuscript includes figures with `\IfFileExists`, so a missing file becomes a labelled placeholder and LaTeX still builds.

The terminal ends with a line like `figures: ['fig_audit_pairs_task1.png', ...]`. That list is the inventory of what this run actually produced.

Figure catalogue

File	What you should see	Needs
`fig_audit_pairs_task1.png` `fig_audit_pairs_task2.png`	Two rows of radiographs: the most similar pairs that sit in different splits, with the correlation printed under each pair. PNG only; these are copies of `results/audit/top_pairs_*.png`.	The audit. Already committed, so Track B copies them.
`fig_icl_shots.pdf`	Four panels. Columns are the two tasks. The top row is test AUROC against the number of in-context examples. The bottom row is balanced accuracy at argmax. Solid lines are raw scores, dashed lines are validation-centred scores. Dotted horizontal lines are the zero-shot models. A thin black line marks chance (0.5).	Final ICL runs in `condition_summary.csv`.
`fig_roc_reliability.pdf`	Top row: ROC curves on the test set for the validation-selected supervised model, the validation-selected VLM, and Gemma-3 zero-shot, using the first seed of each. Bottom row: reliability diagram of the uncalibrated positive-class probability in ten bins. The diagonal is perfect calibration.	At least one of those families present as a final run. With only the committed probes, the supervised curve is drawn and the VLM curves wait.
`fig_efficiency.pdf`	Test AUROC against the number of training images, on a log axis, one panel per task. Series are ResNet-50, Gemma-3 LoRA, and the RAD-DINO probe. Error bars are the standard deviation across seeds when a fraction was repeated. Gemma-3 LoRA is one run per fraction, so it has no error bar. The 100% point comes from the full-data experiment; the other points come from the matching `*_data_efficiency` config (the probe stores every fraction in one config).	At least two fractions for one of those series. The committed RAD-DINO probe is enough to draw its curve.
`fig_robustness.pdf`	Grouped bars of test AUROC under border mask, centre mask, low gamma, high gamma, Gaussian noise, and JPEG quality 10. A centre-masked AUROC far above 0.5 means the model is using something outside the lung fields.	`robustness.csv` from final runs whose config lists perturbations.

> **Opening the files.** The PDF versions are the ones to keep for the paper. The PNG versions are the ones to open in an image viewer. Both are written side by side in `paper/figures/`.

Grad-CAM panels

Grad-CAM is a separate script. It needs a CNN checkpoint from a `--final` run, because it scores the test images. It exits immediately if `meta.json` says the run was validation-only.

```
python scripts/gradcam.py --task task1 --experiment cnn_resnet50 --run seed0
python scripts/gradcam.py --task task2 --experiment cnn_densenet121 --run seed0 --n-show 8
python scripts/gradcam.py --task task1 --experiment cnn_resnet50 --run seed0 --limit 32
```

For every test image it records the fraction of Grad-CAM mass inside the central 15–85% of the frame (about the lung fields). Uniform attention puts 0.49 of the mass there. A mean well below 0.49 means the model is looking at the periphery: labels, markers, collimation.

Outputs, both under `results/gradcam/`:

- `gradcam_{task}_{experiment}_{run}.json` — mean and median centre-mass, the mean within each class, and the fraction of images below the uniform value.
- `gradcam_{task}_{experiment}_{run}.png` — a contact sheet. The top row is the radiograph, the bottom row is the same image with the heatmap overlaid. Titles show the true label, whether argmax was correct, and the centre-mass of that image.

`--n-show` (default 6) is how many examples are drawn, balanced across class and correctness. `--limit` stops the quantitative pass early; leave it at 0 for the number you will report. The next `make_figures.py` copies these PNGs to `paper/figures/fig_gradcam_*.png`.

Run it for the CNN checkpoints you care about. The Colab notebook does seed 0 of `cnn_resnet50`, `cnn_densenet121`, and `cnn_xrv_densenet121` on both tasks. Linear probes and VLMs do not have a Grad-CAM hook in this script.

Rebuild the paper PDF

After tables and figures exist:

```
cd paper && latexmk -pdf main.tex
```

`paper/main.pdf` is git-ignored. The figures it embeds are the files in `paper/figures/`.

11. Build the manuscript tables

```
python scripts/make_tables.py
python scripts/make_tables.py --results results --out paper/generated
```

Every `.tex` file under `paper/generated/` is written by this script. Do not edit those files by hand; the next run overwrites them, and CI rejects a commit in which they do not match `results/`. Missing experiments render as the word “pending”, so the manuscript never contains a number that was typed in.

File	Source
`tab_audit.tex`, `tab_shortcut.tex`	`results/audit/audit_summary.json`
`tab_power.tex`	`results/power/power.json`
`tab_legacy.tex`	`results/legacy/legacy_results.csv`
`tab_main_task1.tex`, `tab_main_task2.tex`	`condition_summary.csv`, raw scores, Youden threshold, test split
`tab_icl.tex`	ICL conditions across *k*
`tab_comparisons.tex`	`comparisons.csv`
`tab_robustness.tex`	`robustness.csv`
`tab_efficiency.tex`	Data-efficiency rows of `condition_summary.csv`
`tab_generation.tex`	`generation_vs_likelihood.csv`
`tab_leakage.tex`	`leakage_sensitivity.csv`
`numbers.tex`	LaTeX macros, including whether any results were available

12. Read a result by hand

These snippets use only pandas, PyArrow, and scikit-learn, which are already installed. Run them from the repository root, for example with `python` and a here-document, or by saving them as a temporary script. They do not write into `results/` or `paper/`.

One run, clean test images

```
import json
from pathlib import Path
import pandas as pd
from sklearn.metrics import roc_auc_score

run = Path("results/task1/probe_imagenet_resnet50/seed0")
meta = json.loads((run / "meta.json").read_text())
df = pd.read_parquet(run / "predictions.parquet")
test = df[(df.split == "test") & (df.perturbation == "none")]
print(meta["experiment"], "final =", meta["final"], "n =", len(test))
print("AUROC", round(roc_auc_score(test.y_true, test.logit), 3))
```

Use a run that exists on disk. After a CNN or VLM `--final` run, point `run` at that directory instead. Validation-only runs have no `split == "test"` rows, so the AUROC line will fail until you pass `--final`.

The summary table the figures use

```
import pandas as pd
cs = pd.read_csv("results/summary/condition_summary.csv")
view = cs[(cs.task == "task1") & (cs.split == "test")
          & (cs.variant == "raw") & (cs.rule == "youden_val")
          & (cs.train_fraction == 1.0) & (cs.shots == -1)]
print(view[["experiment", "auroc_mean", "auroc_std", "n_runs"]]
      .sort_values("auroc_mean", ascending=False)
      .to_string(index=False))
```

A primary comparison

```
import pandas as pd
cmp = pd.read_csv("results/summary/comparisons.csv")
cols = ["task", "id", "tier", "status", "boot_diff", "boot_ci_low",
        "boot_ci_high", "boot_p", "p_holm"]
have = [c for c in cols if c in cmp.columns]
print(cmp.loc[cmp.tier == "primary", have].to_string(index=False))
```

Column names follow `analyze.py`. If a comparison could not be computed, `status` explains which side is missing, and the numeric columns are absent for that row. Quote a primary result only when `status` is `ok`.

Which condition was selected

```
import json
sel = json.loads(open("results/summary/selections.json").read())
print(json.dumps(sel["task1"]["supervised"], indent=2))
print(json.dumps(sel["task1"]["vlm_any"], indent=2))
```

13. Run the GPU suite in Colab

`notebooks/colab_runner.ipynb` is a thin driver around the same scripts. The logic stays in the repository.

1. Runtime → Change runtime type → A100 GPU.
2. Add a secret named `HF_TOKEN`. Accept the MedGemma terms on the model page before the MedGemma cells.
3. Run the cells from the top. The notebook clones the repository, installs `requirements.txt` and `requirements-gpu.txt`, and optionally mirrors `results/` to Google Drive so a runtime reset does not throw away checkpoints. Set `USE_DRIVE = True` for that. The Drive copy is seeded with `rsync --ignore-existing`, so a re-run does not overwrite finished results.
4. The experiment cell loops over the config list in the recommended order. `FINAL = False` is the development setting. Set `FINAL = True` only after `docs/analysis_plan.md` is frozen. Every final run appends to the test-access log inside whichever directory `RESULTS` points at.
5. The next cell runs Grad-CAM for seed 0 of the three CNNs, on both tasks.
6. The last cell runs `analyze.py`, `make_tables.py`, and `make_figures.py` against `RESULTS`, and writes `pip freeze` to `environment-gpu.lock` next to the results. Copy `RESULTS` back into the repository's `results/` and commit the prediction files, metadata, summaries, and figures. Leave `model.pt` and `adapter/` untracked; they are git-ignored.

Re-running the notebook skips finished runs. A validation pass followed by a second pass with `FINAL = True` scores the test split without retraining, as in section 7.

14. Keep the test set honest

- Develop against the validation split. Pass `--final` once, and record it. The access log is the audit trail.
- Do not edit `configs/analysis_plan.yaml` or `docs/analysis_plan.md` after seeing test metrics unless you add a row to the deviations table in the plan: date, change, whether test results had been seen, and why.
- Report every condition that ran, including null results. A crashed seed is rerun with the same seed. A persistent failure is reported as missing, not dropped.
- Do not type numbers into `paper/main.tex`. Regenerate tables and figures.
- Do not point `make_tables.py` or `make_figures.py` at a `SYNTHETIC` tree when `--out` is inside `paper/`. The scripts exit on purpose.
- Smoke runs (`--limit`, `results/_smoke/`) are not results. Their `limit` field keeps them out of the test-set analysis.
- Legacy notebook accuracies stay in `tab_legacy.tex` and are labelled preliminary.

15. Troubleshooting

What you see	What to do
`pytest` cannot import `cxr` or `torch`	Activate `.venv` and install with the CPU commands in section 3. Run pytest from the repository root so `pyproject.toml` is picked up.
`test_vlm_scoring` skipped, "tiny Gemma-3 unavailable"	The processor download failed. Check network access and Hub credentials. The rest of the suite does not need it. Re-run that file alone once the processor is cached.
`FileNotFoundError` mentioning `hparam_selection.json`	`cnn_resnet50_data_efficiency` was started before `cnn_resnet50`. Run the parent config first, at least far enough to write the learning-rate selection.
MedGemma fails with 401 or 403	Accept the model terms in the browser, then export `HF_TOKEN`. Gemma-3 configs do not need that token.
A run prints `skip ... (exists)` and you wanted it redone	Pass `--overwrite`. Without that flag, an existing final run is kept, and an existing validation run is kept unless you are now passing `--final`.
`--final` still seems to be training	The checkpoint is missing. CNN training writes `model.pt`; LoRA writes `adapter/`. If those were deleted, the run trains again, then scores. Embeddings in `cache/` are independent and will be reused.
`analyze.py` reports runs, but `comparisons.csv` rows say `missing conditions` or `missing test predictions`	Those experiments have no `--final` predictions yet, or only a `--limit` smoke run. Check `meta.json` for `"final": true` and `"limit": 0`.
`make_figures.py` prints a short file list	Expected until the matching experiments exist. Audit copies and any curve with enough committed data still appear. Re-run after `analyze.py`.
`refusing to write figures from SYNTHETIC results into paper/`	The results directory contains a `SYNTHETIC` marker. Write to another `--out`, or analyse the real `results/` tree.
CI fails on `git diff --exit-code paper/generated`	Run `python scripts/make_tables.py` and commit the `.tex` files it changed.
Grad-CAM says to run with `--final` first	The named run has no test-set predictions. Re-run that config with `--final`. The checkpoint is reused.
Out of GPU memory on a VLM config	The runner keeps one model in memory and clears it when the checkpoint changes. Close other processes. The published configs assume an A100. Do not switch on 4-bit loading as a silent change to a finished comparison; if you must, record it as a deviation.
Figures exist but the LaTeX PDF has empty boxes	The figure files are missing the exact stem the manuscript asks for (`fig_roc_reliability`, `fig_icl_shots`, `fig_efficiency`, `fig_robustness`, `fig_audit_pairs_task1`, `fig_audit_pairs_task2`). Run `make_figures.py` with `--out paper/figures` and rebuild with `latexmk -pdf main.tex`.

A short map of who writes what

You want	Run	Read
Confidence the code is intact	`pytest -q` and `ruff check src scripts tests`	The pytest summary line
Confidence the graphs can be built	Section 5, output under `/tmp`	`/tmp/synth_figs` and `/tmp/synth/summary`
Duplicate and shortcut findings	`scripts/audit_dataset.py`	`results/audit/`
What the sample size can resolve	`scripts/power.py`	`results/power/`
Per-image scores	`scripts/run_experiment.py --final`	`results/<task>/<experiment>/<run>/predictions.parquet`
Means, intervals, tests	`scripts/analyze.py`	`results/summary/`
Paper figures	`scripts/make_figures.py`	`paper/figures/*.pdf` and `*.png`
Localisation panels	`scripts/gradcam.py`	`results/gradcam/`
Paper tables	`scripts/make_tables.py`	`paper/generated/*.tex`

Source for this guide: `docs/user_guide.html`. Regenerate the PDF from the repository root with headless Chrome, printing to `docs/user_guide.pdf` with headers and footers disabled. Page margins and page breaks are in the stylesheet.